

Yung-Pin Cheng
National Central University

Source Code Quality

1

1

OK · now what?

- 軟體有抽象的本質 (Software has essence of being abstract)
- 軟體有看不到的本質 (Software has essence of being invisible)
- 軟體有難以描述的本質 (software has essence of being hard to describe)
- 請問軟體開發有藝術的成分在嗎? (software has essence of art?)

2

2

Essence of **Invisibility**?



- Can you easily evaluate the progress and quality of painting a wall?
- Can you easily evaluate what a programming is doing?

3

3

Code Review (程式碼鑑賞)

- Software Engineering studies show that
 - The quality of the code between a peak performer and an mediocre programmer is an order of magnitude (10)
- 軟工的研究顯示
 - 高手與庸手的表現有極大的差異，而且往往是一個數量級的差異 (an order of magnitude, 10^1)
- - SACKMAN, EROKSON, and GRANT
- Another Study shows that
 - The number of years in experience is not a key factor to peak performer and mediocre programmer.
 - 另外一個研究顯示
 - 成為高手與庸手與年資經驗沒有絕對關係

4

4

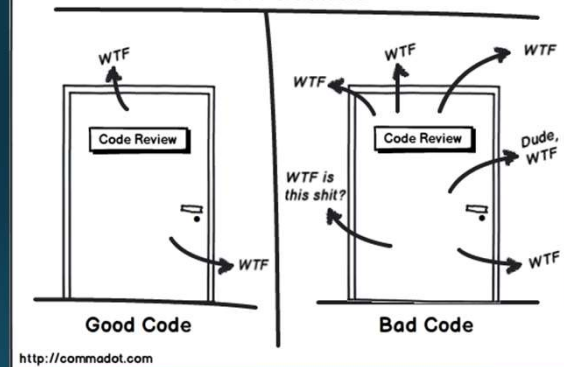
Code Review

- A process to check and examine submitted code
 - Before the integration (sometimes called code inspection)
 - After the test (mostly is called code review)

5

5

Code Quality Measurement: WTFs/Minute



6

6

Yes

- Do you think code contains ART?
- What is the difference between ART and Science?
 - ART – no objective criteria
 - Science – based on objective criteria
 - Repeatable experiments
 - Describable by explicit formalism such as maths
- How do you judge a painting or art pieces?
- 我們認為寫軟體有藝術的成分在
- 藝術的東西
 - 有時只能意會不能言傳
- 有學過畫畫等藝術的東西嗎？請問你怎麼學的？
- 上過藝術鑑賞這種課程嗎？
- 通常藝術的傳授一定會配合藝術的鑑賞

7

7

AND ?

- How many good code have you ever read? (excluding the small fragments from text books) 你看過多少別人的程式碼？（教科書的片段程式碼不算）
- What is a good code? (你知道什麼樣的程式碼是好的？)
- What is a bad code? (你知道什麼樣的程式碼是壞的？)
- Let's look at the bad code first (我們先來看看壞的)

8

8

Typical symptoms of bad code

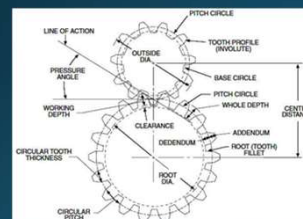
- Difficult to understand
- **Difficult to change (changing at one place results in changes in many other places)**
- Hardwire the solutions (for example, instead of coming out a nice algorithms and data structures, many **if** is used)
- Source code is tightly coupled because the excessive use of global variables.
- Poor design that is difficult to extend.

9

9

Reasons for writing bad code

- CS curriculum
 - In CS major, main focuses are problem solving such as algorithms, techniques (networking, multimedia).
 - writing high quality code is not encouraged or viewed as an important topic among students

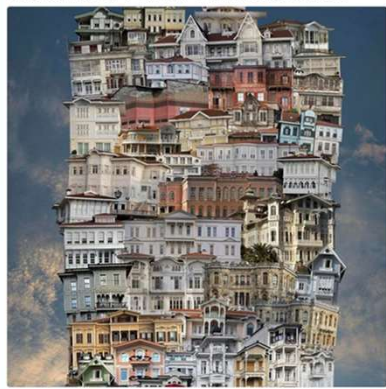


We teach students how to assemble gears to solve a problem but do not educate them how to produce quality gears and measure quality of the gears

10

10

如果某个软件系统（代码）给你留下的是如下图这样的感受，相信我，这已经是你有机会看到的软件系统（代码）中做得比较好的了。



11

11

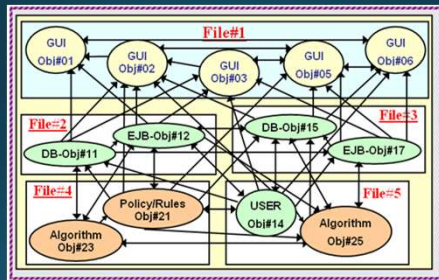
What is a good design?



12

12

Comfortable?



13

13

The truths are

- Clean and simple interface, replaceability, modularity, composability

Their importance is not obvious in Software development, particularly if you never has chance to maintain a software

14

14

Coupling (耦合)

- Most software engineering methods (e.g., OO) focus on create beautiful design so that
 - Change one thing does not involving changing many other things
 - Replace/substitute one thing does not involving modifying many other things
 - Extend one thing does not involving rewriting many other things
- High coupling is bad
- We pursuit low coupling
- OO is a technique to **decouple** the system architecture
- The concept of coupling will be introduced in the future.

Remember this word

15

15

Again

- Why writing low-coupled source code is not as easy as other engineering?
 - Software is invisible and **source code is not geometric related that you can judge from your eyes**
 - The dependency and coupling are not obvious and easy to identify. Consider a poor programmer writing thousand lines of code for you.
 - Software behaviors are dynamic

16

16

Reasons for writing bad code

- Experienced programmers may write bad code
 - Tight schedule and limited budget
 - Demo-oriented programming
 - Not enough time to do design
 - They accustomed themselves to write bad code if no adequate software engineering processes has been established in their working environment

17

17

Motivation

- What is a bad code smell?
 - No formal definitions and descriptions have been established so far
 - Sometimes, it is not easy to explain why a piece of code is bad to beginners
 - This means "bad" is not measurable quantitatively
 - Understand what is bad can lead to a better understanding of what is good. For example, why using some design pattern is good and what can be improved?

18

18

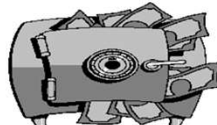
Spaghetti Code

Development AntiPattern: Spaghetti Code

spaghetti code [Slang] an undocumented piece of software source code that cannot be extended or modified without extreme difficulty due to its convoluted structure.



Un-structured code
is a liability



Well structured code
is an investment.

MITRE

19

19



20

20

Development AntiPattern: Spaghetti Code

Symptoms

- Quick demonstration code that became operational
- "Lone Ranger" programmer (*who was that masked man?*)
- Obsolete or scanty documentation
- 50% of maintenance spent on system rediscovery
- Hesitant Programmer Syndrome
 - More likely to break it than extend it
 - Easier to just rewrite it
- Cannot be reused
 - System software and COTS packages can't be upgraded
 - Performance cannot be optimized
- User work arounds

MITRE

21

21

Development AntiPattern: Spaghetti Code

Object-Oriented Spaghetti Code

- Many object methods with no parameters
- Suspicious class or global variables
- Intertwined and unforeseen relationships between objects
- Process-oriented methods, objects with process-oriented names.
- OO advantage lost – inheritance cannot be used to extend the system, polymorphism not effective either.

Bottom Line: Software has reached point of diminishing returns here the effort involved to maintain existing code exceeds the cost of developing a new "ground up" solution

MITRE

22

22

Lasagna code

- Lasagna code is used to describe software that has a simple, understandable, and layered structure. Lasagna code, although structured, is unfortunately monolithic and not easy to modify. An attempt to change one layer conceptually simple, is often very difficult in actual practice.

23

23



24

24

Ravioli code

- In ravioli code, each of the components, or objects, is a package containing some meat or other nourishment for the system; any component can be modified or replaced without significantly affecting other components.
- The ideal software structure is one having components that are small and loosely coupled; this ideal structure is called ravioli code.

25

25



26

26

Now, let's look at some spaghetti code and give a critics

27

27

Copy-Paste Programming

Or, if you prefer, guns don't kill people, *people* kill people. Just make sure that source code isn't pointed at me when it goes off. There are always risks. When you copy and paste code, vigilance is required to make sure you (or someone you work with) isn't falling into the trap of [copy and paste code duplication](#):



28

28

Copy-Paste Programming

- Undoubtedly the most popular reason for creating a routine is to avoid duplicate code. Similar code in two routines is a warning sign. **David Parnas says that if you use copy and paste while you're coding, you're probably committing a design error.** Instead of copying code, move it into its own routine. Future modifications will be easier because you will need to modify the code in only one location. The code will be more reliable because you will have only one place in which to be sure that the code is correct.

29

29

Top 100 signs that I may be writing spaghetti code

1. I have no idea where this constant is defined
 2. I have printf stmts littered throughout my code.
 3. There is an error but it isn't handled, and I can't find it to figure out what's wrong.
 4. Passing variables directly to 635 functions and using 'global' on 453 functions
 5. I have to modify the same function 10 times because I have not moved it into a function...yet.
 6. I find that rewriting is quicker and easier than modifying the original design.
 7. You have a library of small functions that you wrote yourself, and will cut'n'paste them into almost every project you work on... Whether they're needed or not...
- Now let's continue and win your credits

30

30

Top 100 signs that I may be writing spaghetti code- (CONT)

8. Every time you want to change a variable you have to use grep to find it ;)
9. you have so many "for" loops you've declared a variable for each letter in the alphabet.
10. there are enough "if" statements to make you scroll to the right.
11. You see comments in the code that read something like:
[Your Name: Don't ask me how or why this works, I know it looks overly complex and it appears there IS a simpler solution, but this works, it fixed a bug from the 'simpler code'...just leave it be...don't touch it, I've hired a voodoo witch doctor to place a curse on anybody who alters this code...trust me, just leave it alone!]
12. you know you're digging through a bowl of the stuff when, as a regular tool, you're using the window's search feature to find functions, variables and other bits of code.

31

31

Top 100 signs that I may be writing spaghetti code- (CONT)

13. having to "peel" your way through while/for loops and a chunks of IF and ELSE statements because the original author (probably in all his haste) couldn't be bothered to use brackets. The code works but not always for the right reasons.
14. use every imaginable variation of the word "foobar" in variable names
15. You sit down, open your program, and wonder where to begin with debugging... (ooh boy, I know the feeling of that!)
16. You wrap it up and realize that there are 20 funtions in the include files which are never called, although at some point, they were all required.
17. Changing database is (e.g MS SQL to MySQL) requires the rewrite of SQL statments.
18. You realize that you are changing global variables with functions inside include files
19. You create a separate class for every single function.
20. You have no idea what a CRC Card is ...

32

32

- To be continued!!

33

33

Coding Style

- Anything wrong?

This fragment of code comes from a large program written many years ago:

```
if ( (country == SING) || (country == BRNI) ||
    (country == POL) || (country == ITALY) )
{
    /*
     * If the country is Singapore, Brunei or Poland
     * then the current time is the answer time
     * rather than the off hook time.
     * Reset answer time and set day of week.
     */
    ...
}
```

34

34

Be consistent

Be consistent. Give related things related names that show their relationship and highlight their difference.
Besides being much too long, the member names in this Java class are wildly inconsistent:

```
? class UserQueue {
?     int noOfItemsInQ, frontOfTheQueue, queueCapacity;
?     public int noOfUsersInQueue() {...}
? }
```

The word "queue" appears as Q, Queue and queue. But since queues can only be accessed from a variable of type UserQueue, member names do not need to mention "queue" at all; context suffices, so

```
? queue.queueCapacity
```

is redundant. This version is better:

```
class UserQueue {
    int nItems, front, capacity;
    public int nUsers() {...}
}
```

since it leads to statements like

```
queue.capacity++;
n = queue.nUsers();
```

35

35

Coding horror!

36

36

What's wrong with this piece of code?



C++ Example of a Low-Quality Routine

```
void HandleStuff( CORP_DATA & inputRec, int crntQtr, EMP_DATA empRec,
double & estmRevenue, double ytdRevenue, int screenX, int screenY,
COLOR_TYPE & newColor, COLOR_TYPE & prevColor, StatusType & status,
int expenseType )
{
    int i;
    for ( i = 0; i < 100; i++ ) {
        inputRec.revenue[i] = 0;
        inputRec.expense[i] = corpExpense[ crntQtr ][ i ];
    }
    updateCorpDatabase( empRec );
    estmRevenue = ytdRevenue * 4.0 / (double) crntQtr;
    newColor = prevColor;
    status = SUCCESS;
    if ( expenseType == 1 ) {
        for ( i = 0; i < 12; i++ )
            profit[i] = revenue[i] - expense.type1[i];
    }
    else if ( expenseType == 2 ) {
        profit[i] = revenue[i] - expense.type2[i];
    }
    else if ( expenseType == 3 )
        profit[i] = revenue[i] - expense.type3[i];
}
```

37

37

- The routine has a bad name. *HandleStuff()* tells you nothing about what the routine does.
- The routine isn't documented. (The subject of documentation extends beyond the boundaries of individual routines and is discussed in Chapter 32, "Self-Documenting Code.")
- The routine has a bad layout. The physical organization of the code on the page gives few hints about its logical organization. Layout strategies are used haphazardly, with different styles in different parts of the routine. Compare the styles where *expenseType == 2* and *expenseType == 3*. (Layout is discussed in Chapter 31, "Layout and Style.")
- The routine's input variable, *inputRec*, is changed. If it's an input variable, its value should not be modified (and in C++ it should be declared *const*). If the value of the variable is supposed to be modified, the variable should not be called *inputRec*.
- The routine reads and writes global variables—it reads from *corpExpense* and writes to *profit*. It should communicate with other routines more directly than by reading and writing global variables.

38

38

- The routine doesn't have a single purpose. It initializes some variables, writes to a database, does some calculations—none of which seem to be related to each other in any way. A routine should have a single, clearly defined purpose.
- The routine doesn't defend itself against bad data. If *crntQtr* equals 0, the expression *ytdRevenue * 4.0 / (double) crntQtr* causes a divide-by-zero error.
- The routine uses several magic numbers: 100, 4.0, 12, 2, and 3. Magic numbers are discussed in Section 12.1, "Numbers in General."
- Some of the routine's parameters are unused: *screenX* and *screenY* are not referenced within the routine.
- One of the routine's parameters is passed incorrectly: *prevColor* is labeled as a reference parameter (&) even though it isn't assigned a value within the routine.
- The routine has too many parameters. The upper limit for an understandable number of parameters is about 7; this routine has 11. The parameters are laid out in such an unreadable way that most people wouldn't try to examine them closely or even count them.
- The routine's parameters are poorly ordered and are not documented. (Parameter ordering is discussed in this chapter. Documentation is discussed in Chapter 32.)

39

39

Curly's law

- Curly's Law is about choosing a single, clearly defined goal for any particular bit of code: **Do One Thing**. That much is clear. But in choosing one thing, you are ruling out an infinite universe of other possible things you *could* have done.
- **Curly's Law also means consciously choosing what your code won't do.** This is much more difficult than choosing what to do, because it runs counter to all the natural generalist tendencies of software developers. It could mean breaking code apart, violating traditional OOP rules, or introducing duplicate code. It's taking one step backward to go two steps forward.

40

40

level of abstraction in the code



```
class EmployeeCensus: public ListContainer {
public:
    ...
    // public routines
    void AddEmployee( Employee employee );
    void RemoveEmployee( Employee employee );

    Employee NextItemInList();
    Employee FirstItem();
    Employee LastItem();
    ...
private:
    ...
};
```

41

41

- This class is representing two ADTs: an employee and a ListContainer.

C++ Example of a Class Interface with Consistent Levels of Abstraction

```
class EmployeeCensus {
public:
    ...
    // public routines
    void AddEmployee( Employee employee );
    void RemoveEmployee( Employee employee );
    Employee NextEmployee();
    Employee FirstEmployee();
    Employee LastEmployee();
    ...
private:
    ListContainer m_EmployeeList;
    ...
};
```

The abstraction of all these routines is now at the "employee" level.

That the class uses the ListContainer library is now hidden.

42

42

EVOLVES TO LOOK LIKE THIS:



C++ Example of a Class Interface That's Eroding Under Maintenance

```

class Employee {
public:
    ...
    // public routines
    FullName GetName() const;
    Address GetAddress() const;
    PhoneNumber GetWorkPhone() const;
    ...
    bool IsJobClassificationValid( JobClassification jobClass );
    bool IsZipCodeValid( Address address );
    bool IsPhoneNumValid( PhoneNumber phoneNumber );

    SqlQuery GetQueryToCreateNewEmployee() const;
    SqlQuery GetQueryToModifyEmployee() const;
    SqlQuery GetQueryToRetrieveEmployee() const;
    ...
private:
    ...
};
  
```

43

43

Curly's Law for good coding abstraction

In *Outliving the Great Variable Shortage*, Tim Ottinger invokes Curly's Law:

A variable should mean one thing, and one thing only. It should not mean one thing in one circumstance, and carry a different value from a different domain some other time. It should not mean two things at once. It must not be both a floor polish and a dessert topping. It should mean One Thing, and should mean it all of the time.

The late, great Jack Palance played grizzled cowboy Curly Washburn in the 1991 comedy *City Slickers*. Curly's Law is defined in this bit of dialog from the movie:

Curly: Do you know what the secret of life is?

Curly: This. [holds up one finger]

Mitch: Your finger?

Curly: One thing. Just one thing. You stick to that and the rest don't mean shit.

Mitch: But what is the "one thing?"

Curly: [smiles] That's what you have to find out.



44

44

Curly's Law

Curly's Law, *Do One Thing*, is reflected in several core principles of modern software development:

- Don't Repeat Yourself**
If you have more than one way to express the same thing, at some point the two or three different representations will most likely fall out of step with each other. Even if they don't, you're guaranteeing yourself the headache of maintaining them in parallel whenever a change occurs. And change *will* occur. Don't repeat yourself is important if you want flexible and maintainable software.
- Once and Only Once**
Each and every declaration of behavior should occur once, and only once. This is one of the main goals, if not the main goal, when refactoring code. The design goal is to eliminate duplicated declarations of behavior, typically by merging them or replacing multiple similar implementations with a unifying abstraction.
- Single Point of Truth**
Repetition leads to inconsistency and code that is subtly broken, because you changed only some repetitions when you needed to change all of them. Often, it also means that you haven't properly thought through the organization of your code. Any time you see duplicate code, that's a danger sign. Complexity is a cost; don't pay it twice.

45

45

Although Curly's Law definitely applies to normalization and removing redundancies, *Do One Thing* is more nuanced than the various restatements of *Do Each Thing Once* outlined above. It runs deeper. Bob Martin refers to it as [The Single Responsibility Principle](#):

The Single Responsibility Principle says that a class should have one, and only one, reason to change. As an example, imagine the following class:

```

class Employee
{
    public Money calculatePay()
    public void save()
    public String reportHours()
}
  
```

This class violates the SRP because it has three reasons to change:

1. The business rules having to do with calculating pay.
2. The database schema.
3. The format of the string that reports hours.

We don't want a single class to be impacted by these three completely different forces. We don't want to modify the Employee class every time the accounts decide to change the format of the hourly report, or every time the DBAs make a change to the database schema, as well as every time the managers change the payroll calculation. Rather, we want to separate these functions out into different classes so that they can change independently of each other.

46

46

Copy-paste code from internet?

Per Mr. Bambrick, copy and pasted code from the internet is good because:

- Code stored on blogs, forums, and the web in general is very easy to find.
- You can inspect the code before you use it.
- Comments on blogs give some small level of feedback that might improve quality.
- Pagerank means that you're more likely to find code that might be higher quality.
- Code that is easy to read and understand will be copied and pasted more, leading to a sort of viral reproductive dominance.
- The programmer's ego may drive her to only publish code that she believes is of sufficient quality.

But copy and pasted code from the internet is bad because:

- If the author improves the code, you're not likely to get those benefits.
- If you improve the code, you're not likely to pass those improvements back to the author.
- Code may be blindly copied and pasted without understanding what the code actually does.
- Pagerank doesn't address the quality of the code, or its fitness for your purpose.
- Code is often 'demo code' and may purposely gloss over important concerns like error handling, sql injection, encoding, security, etc.

47

47

Code Smell

Comments	There's a fine line between comments that illuminate and comments that obscure. Are the comments necessary? Do they explain "why" and not "what"? Can you refactor the code so the comments aren't required? And remember, you're writing comments for people, not machines.
Long Method	All other things being equal, a shorter method is easier to read, easier to understand, and easier to troubleshoot. Refactor long methods into smaller methods if you can.
Long Parameter List	The more parameters a method has, the more complex it is. Limit the number of parameters you need in a given method, or use an object to combine the parameters.
Duplicated code	Duplicated code is the bane of software development. Stamp out duplication whenever possible. You should always be on the lookout for more subtle cases of near-duplication, too. Don't Repeat Yourself!
Conditional Complexity	Watch out for large conditional logic blocks, particularly blocks that tend to grow larger or change significantly over time. Consider alternative object-oriented approaches such as decorator, strategy, or state.
Combinatorial Explosion	You have lots of code that does <i>almost</i> the same thing.. but with tiny variations in data or behavior. This can be difficult to refactor-- perhaps using generics or an interpreter?

48

48

Large Class	Large classes, like long methods, are difficult to read, understand, and troubleshoot. Does the class contain too many responsibilities? Can the large class be restructured or broken into smaller classes?
Type Embedded in Name	Avoid placing types in method names; it's not only redundant, but it forces you to change the name if the type changes.
Uncommunicative Name	Does the name of the method succinctly describe what that method does? Could you read the method's name to another developer and have them explain to you what it does? If not, rename it or rewrite it.
Inconsistent Names	Pick a set of standard terminology and stick to it throughout your methods. For example, if you have <code>Open()</code> , you should probably have <code>Close()</code> .
Dead Code	Ruthlessly delete code that isn't being used. That's why we have source control systems!
Speculative Generality	Write code to solve today's problems, and worry about tomorrow's problems when they actually materialize. Everyone loses in the "what if.." school of design. <i>You (Probably) Aren't Gonna Need It.</i>
Oddball Solution	There should only be one way of solving the same problem in your code. If you find an oddball solution, it could be a case of poorly duplicated code - or it could be an argument for the adapter model, if you really need <i>that</i> solution to the problem.

49

49

Code Smells Between Classes	
Alternative Classes with Different Interfaces	If two classes are similar on the inside, but different on the outside, perhaps they can be modified to share a common interface.
Primitive Obsession	Don't use a gaggle of primitive data type variables as a poor man's substitute for a class. If your data type is sufficiently complex, write a class to represent it.
Data Class	Avoid classes that passively store data. Classes should contain data <i>and</i> methods to operate on that data, too.
Data Clumps	If you always see the same data hanging around together, maybe it belongs together. Consider rolling the related data up into a larger class.
Refused Bequest	If you inherit from a class, but never use any of the inherited functionality, should you really be using inheritance?
Inappropriate Intimacy	Watch out for classes that spend too much time together, or classes that interface in inappropriate ways. Classes should know as little as possible about each other.

50

50

Indecent Exposure	Beware of classes that unnecessarily expose their internals. Aggressively refactor classes to minimize their public surface. You should have a compelling reason for every item you make public. If you don't, hide it.
Feature Envy	Methods that make extensive use of another class may belong in another class. Consider moving this method to the class it is so envious of.
Lazy Class	Classes should pull their weight. Every additional class increases the complexity of a project. If you have a class that isn't doing enough to pay for itself, can it be collapsed or combined into another class?
Message Chains	Watch out for long sequences of method calls or temporary variables to get routine data. Intermediaries are dependencies in disguise.
Middle Man	If a class is delegating all its work, why does it exist? Cut out the middleman. Beware classes that are merely wrappers over other classes or existing functionality in the framework.
Divergent Change	If, over time, you make changes to a class that touch completely different parts of the class, it may contain too much unrelated functionality. Consider isolating the parts that changed in another class.
Shotgun Surgery	If a change in one class requires cascading changes in several related classes, consider refactoring so that the changes are limited to a single class.

51

51

Shotgun Surgery	If a change in one class requires cascading changes in several related classes, consider refactoring so that the changes are limited to a single class.
Parallel Inheritance Hierarchies	Every time you make a subclass of one class, you must also make a subclass of another. Consider folding the hierarchy into a single class.
Incomplete Library Class	We need a method that's missing from the library, but we're unwilling or unable to change the library to include the method. The method ends up tacked on to some other class. If you can't modify the library, consider isolating the method.
Solution Sprawl	If it takes five classes to do anything useful, you might have solution sprawl. Consider simplifying and consolidating your design.

52

52

Other smells

- Non-localized plans
- Too many bugs
- Too hard to understand
- Too hard to change

53

53

Non-localized plan

- Adding a feature requires a plan. If adding a feature requires changing many parts of a program, it is called a "non-localized plan".
- Parallel class hierarchies: adding a class in one class hierarchy requires adding a class in another
- Example: a new item class requires a new itemFactory class

54

54

How to refactor non-localized plan

- Make a new object that represents everything that changes
- Methods that change together should stay together
- Many patterns address this kind of problem

55

55

Too many bugs

- If one part of the system has more than its share of the bugs, there is probably a good reason.
- Redesign, rewrite, refactor

56

56

Too hard to understand

- Hard to fix bugs because you don't understand
- Hard to change because you don't understand

57

57

Too hard to change

- Because of lack of tests
- Because of dependencies
 - Global variables
 - Very large modules
 - Importing too many classes
- Because of duplication or non-localized plans

58

58